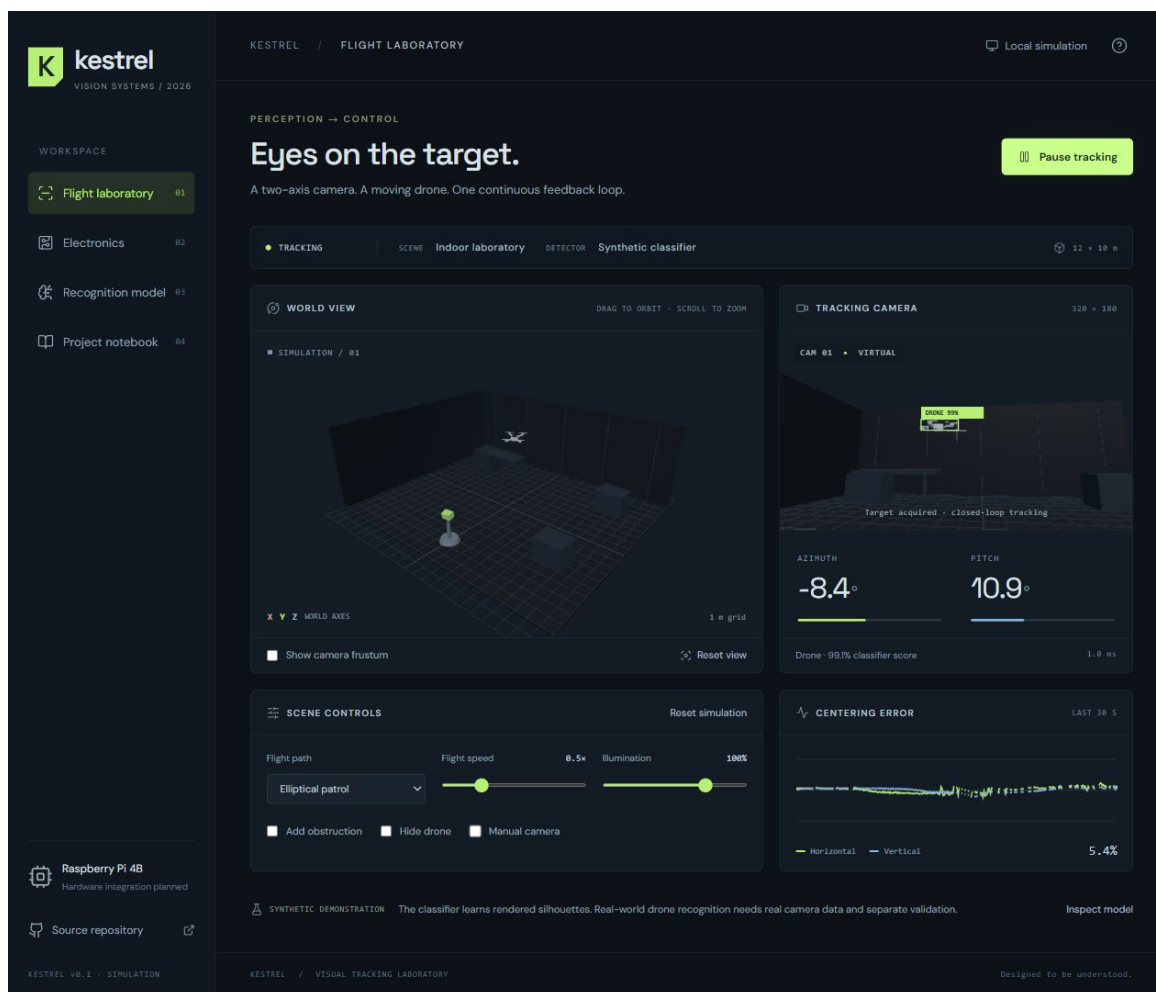


Kestrel

Drone tracking simulator and proposed electronics

Technical project dossier

A browser-based engineering demonstration that learns the appearance of a procedural drone, follows it with a virtual pan/tilt camera, and explains the proposed Raspberry Pi electronics.



Release boundary: synthetic demonstration and proposed electronics. Real-camera recognition, wiring and motor actuation are not validated.

Source: github.com/hugowalker/unravel

Perception and control

The simulation renders a room and quadcopter in Three.js. Its tracking camera captures 320 x 180 RGB images. Runtime detection receives pixels only; it does not read the target position from the scene.



Browser silhouette classifier

720 rendered crops are balanced between drone views and geometric non-drone objects. A fixed seed assigns 576 crops to training and 144 to validation. Views vary yaw, pitch, roll and illumination. The same procedural assets appear in both splits, so the score measures a narrow synthetic task.

Bright connected regions become candidates. Small raster gaps are joined, while the original silhouette supplies 16 x 16 occupancy features plus aspect ratio. Full-batch gradient descent fits a logistic classifier. Model weights can be exported from the app.

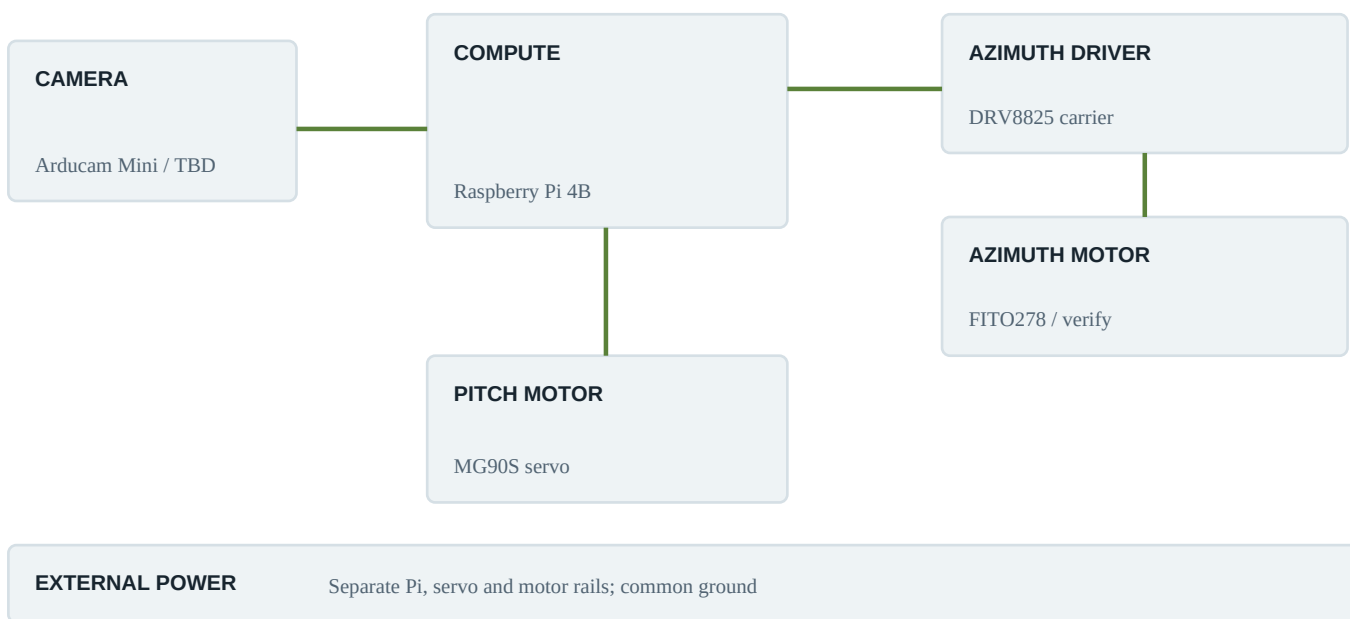
State machine and controller

State / setting	Behavior
SEARCHING	Bounded azimuth sweep with several pitch rows.
ACQUIRING	Require three repeated detections before tracking.
TRACKING	Steer from bounding-box centre error; 2.5% deadband on each axis.
LOST	Hold briefly, then resume searching after 0.65 seconds.
Motion limits	42 degrees/s azimuth; 30 degrees/s pitch.
Simulated travel	Azimuth -160 to +160 degrees; pitch -15 to +55 degrees.
MANUAL / IDLE	Manual sliders or paused motion.

These motion values are demonstration settings. Physical limits depend on the actual mechanism, cabling, driver and calibration.

Electrical architecture

The app provides a selectable component diagram, proposed connection table, and downloadable SVG. Revision A is an architecture proposal; the exact camera module, motor ratings and supplies still need verification.



Supplies and signal boundaries

Use a dedicated 5.1 V USB-C supply for the Pi, an external regulated 4.8-5.0 V servo rail, and a separate current-limited motor rail. The DRV8825 supports 8.2-45 V at VMOT; 12 V is only a candidate until the motor and current limit are verified. GPIO carries 3.3 V logic, not motor power.

Place at least 47 uF bulk capacitance near VMOT. Set the driver current limit using the actual motor rating and carrier sense resistors. Use a common ground reference, suitable fusing, strain relief and a physical motor-power disconnect. Do not join positive supply rails.

Open hardware questions

Confirm the Arducam SKU before assigning SPI or CSI connections. Verify the stepper identifier, coil pairs, phase current, step angle and gearing. Confirm the servo variant, stall-current demand and PWM compatibility. Add homing before autonomous physical scanning.

Proposed wiring and integration

Physical pin	BCM / rail	Destination
11	GPIO17	DRV8825 STEP
13	GPIO27	DRV8825 DIR
15	GPIO22	DRV8825 nENBL; 10 kohm pull-up to 3.3 V
12	GPIO18	MG90S PWM; verify signal compatibility
1	3V3	DRV8825 nRESET and nSLEEP held high
6	GND	Common signal / power reference
Unassigned	Camera bus	Verify exact module before wiring

Driver details

M0, M1 and M2 low selects full-step mode. Keep nENBL high until the mechanism is homed. Identify the two motor coil pairs with power disconnected: A1/A2 connect to one pair, B1/B2 to the other. Do not infer pairs from wire colours. The overview schematic groups connections; the component inspector and hardware guide describe the passive parts and enable logic.

Physical integration sequence

- Identify the exact camera, motor and carrier variants. Verify supply voltages before connecting loads.
- Set phase-current limiting; establish unloaded servo neutral and travel endpoints.
- Install and test a home switch. Measure steps per degree, gearing and direction signs.
- Train and benchmark a real-image detector on the Pi using the actual capture interface.
- Implement the physical MotorInterface with deterministic step timing, stale-frame stops and calibrated travel.
- Validate first with a stationary or hand-moved target.

Shipped motor adapter: DryRunMotors only records bounded commands. It never imports GPIO or energises a motor. The OpenCV camera dry-run supports an existing capture device or file; it is not a generic SPI driver.

Data, software and deployment

Run the browser laboratory

Install Node.js 22.12+ and pnpm 11+. From the repository root:

```
pnpm install
pnpm dev

pnpm build
pnpm preview
```

The default demonstration is static and local. It requires WebGL but no Python service. Select Start tracking to train the synthetic model. The model is cached on the current device. Use the model page to retrain or export it.

Move toward real images

Candidate sources include a downloadable DJI Mavic 2 Pro photogrammetry model on Sketchfab, a labelled Drone Detection dataset on Roboflow, and Amateur Unmanned Air Vehicle Detection on Kaggle. These were located, not incorporated. Verify licences and provenance before using or redistributing any asset. Source links are in docs/dataset-guide.md.

Add real images from the intended room and working distances, including empty scenes. Label the drone itself. Split by recording, not by adjacent frames. Keep the final real test set separate from model selection.

```
python -m pip install -r backend/requirements-ml.txt
python scripts/train_detector.py --data data/drone/data.yaml
python scripts/export_detector.py runs/kestrel/weights/best.pt
```

Local detector interface

Set KESTREL_MODEL to a trained local checkpoint and run the FastAPI service on 127.0.0.1:8000. The browser sends JPEG/PNG frames and frame IDs over /ws/detect. The service returns normalised xywh boxes and scores for the drone class. A missing model is reported explicitly; the service never silently downloads weights.

The browser permits one outstanding frame, discards responses older than 400 ms and disconnects after a three-second timeout. The service has no motor access. Optional Ultralytics dependencies have their own AGPL/commercial licensing requirements; see THIRD_PARTY.md.

Evidence and next milestones

Automated verification

Production build and TypeScript checks passed. Eight TypeScript tests and five Python tests cover control states, bounds, classifier fitting, image validation, stale input and dry-run motor behavior. Browser checks of the 6 October version verified navigation, training, tracking, component inspection and diagram export.

One recorded synthetic patrol

Measurement	Result
Samples / simulated flight time	447 / 23.27 seconds at 0.5x flight speed
TRACKING / ACQUIRING / LOST	352 / 59 / 36 samples
Tracked after settling (2.5 scene seconds)	309 samples
Within 10% error on each axis, tracked subset	309 of 309 (100%)
Mean pixel-detector processing time	1.19 ms; excludes rendering and capture
Held-out synthetic crop accuracy	100% on 144 same-asset crops

The centring criterion was met while tracking, but tracking was not continuous: 78.7% of samples were in TRACKING. Brief classifier dropouts are a known limitation. The synthetic validation score does not establish performance on unseen drone shapes, bright clutter, real cameras or physical motion. Raw measurements are preserved in docs/validation-session.json.

Remaining validation

Real-image precision/recall and false positives; Raspberry Pi throughput; camera compatibility; calibrated motor motion; homing and stop behavior; long-duration and occlusion testing. No physical wiring or flying-drone demonstration has been validated by this release.

Manufacturer and implementation references

- DRV8825 carrier documentation
- Raspberry Pi hardware
- TowerPro MG90S
- Ultralytics Raspberry Pi deployment
- Three.js